Restaurant POS Frontend Development Guide

Implementation roadmap aligned to the completed backend

Project context	Restaurant POS system with sales, tables, reservations, kitchen, reports, coupons, and refunds already available in the backend.
Audience	Frontend developer building the React + Tailwind POS UI.
Goal	Follow the exact build order so the frontend matches backend capabilities without rework.

Recommended stack: React + Vite + TypeScript + Tailwind CSS + Axios + React Router + Zustand

1. Purpose of this guide

This guide explains how to build the frontend in the correct sequence based on the backend that is already implemented. It is written so you can stop coding randomly and build each screen, API integration, and workflow in a safe order.

The main rule is simple: do not move to the next phase until the current phase works end to end in the browser and in Postman. That prevents duplicate work and keeps the UI aligned with backend behavior.

Backend capabilities already available

- Authentication and role-based access
- Products, inventory, low-stock logic, and adjustments
- Sales for takeaway and table sessions
- Partial and full payments
- Invoice endpoint
- Kitchen queue and dashboard
- Tables and active table sessions
- Reservations linked to sales
- Coupons, discounts, refunds, and reports

Frontend success criteria

- Cashier can log in and work from a single POS screen.
- Products load from the backend and can be added to a cart.
- User can create takeaway sales and table sales.
- Occupied tables can be reopened and continued.
- Payments, invoices, reservations, and kitchen status are visible in the UI.

2. Correct frontend build order

Phase	What to build	Why it comes here	Stop and test before next step
1	Project setup	Creates the base app and styling system.	App runs in browser.
2	Auth foundation	Everything else depends on login and token handling.	Login works and protected route opens.
3	POS shell UI	Gives a stable layout before adding logic.	Three-column layout renders cleanly.
4	Products API + grid	Products are the first real backend data	Products load from backend.

		source.	
5	Cart system	All sale flows depend on cart state.	Add/remove/qty/totals work.
6	Create takeaway sale	First complete business flow.	Cart creates completed sale.
7	Table UI	Needed for dine-in flow.	Tables load and can be selected.
8	Active table session	Needed to continue occupied orders.	Existing table sale loads correctly.
9	Payment flow	Needed for open and partial payments.	Partial and final payments work.
10	Invoice / receipt view	Cashier needs a final bill view.	Invoice endpoint renders clearly.
11	Reservations UI	Needed after table flow is stable.	Reservations can be created and seated.
12	Coupons / discounts	Pricing controls come after sale flow is stable.	Discounted totals match backend.
13	Refund UI	Comes after invoice and payment logic.	Refund updates sale and stock.
14	Kitchen status + realtime	Operational enhancement after core POS works.	Kitchen data updates in UI.
15	Reports / dashboard	Admin and analytics layer last.	Report pages load valid data.

Important: Do not build payments before cart, do not build reservations before table flow, and do not build analytics before the core cashier journey works.

3. Frontend foundation

Backend endpoints used

Endpoint	Purpose
POST /api/auth/login	Authenticate and return token.

GET /api/products	Used later to prove token handling is working.

Implementation steps

1. Create the Vite React TypeScript project.
2. Install Tailwind CSS, Axios, React Router, and Zustand.
3. Create axios.ts with the backend base URL and bearer-token interceptor.
4. Create auth store for token, user, login, and logout actions.
5. Create App.tsx with a protected route around the POS page.
6. Create the Tailwind login page and the base POS shell page.

Test checkpoint

- App starts without build errors.
- Login page loads.
- Protected /pos route opens only after login.

Notes

- Keep Tailwind styles clean and readable from the start. Do not use inline styles once Tailwind is installed.

4. Products and catalog browsing

Backend endpoints used

Endpoint	Purpose
GET /api/products	Load product list for the main POS grid.

Implementation steps

7. Create products.api.ts.
8. Load products inside PosPage.
9. Render products as cards with name, price, category, and low-stock badge.
10. Add search input and category UI placeholders. Filtering can be frontend-only at first.

Test checkpoint

- Products appear on the POS screen.
- No console errors from product loading.
- Search input is visible even if filtering is not complete yet.

5. Cart system

Implementation steps

11. Create cart.store.ts with items, addItem, increaseQty, decreaseQty, removeItem, clearCart, subtotal, taxTotal, and grandTotal.
12. Connect the product card Add button to the cart store.
13. Render cart items in the right panel.
14. Show quantity controls, remove button, and totals section.

Test checkpoint

- Cart updates when Add is clicked.
- Increasing and decreasing quantity works.
- Removing items and clearing cart works.
- Totals change correctly with quantity and tax rate.

Notes

- The cart is the last shared state you should harden before creating sales. Do not move on until it is reliable.

6. Takeaway sale creation

Backend endpoints used

Endpoint	Purpose
POST /api/sales	Create a takeaway sale when no table is selected.

Implementation steps

15. Create sales.api.ts with createSale().
16. On Create Sale click, convert cart items into backend payload format.
17. For takeaway mode, send paymentMethod such as CASH.
18. After successful response, clear the cart and show success feedback.

Test checkpoint

- Backend sale is created successfully.
- Cart clears after success.
- Frontend shows an error message if backend returns 400/401.

Notes

- If a sale fails, inspect the backend message first. Common causes are missing paymentMethod or no open shift.

7. Table UI for dine-in

Backend endpoints used

Endpoint	Purpose
GET /api/tables	Load table list and active table status.
POST /api/sales	Create a table-linked sale using tableId.

Implementation steps

19. Create tables.api.ts with getTables().
20. Load tables alongside products in PosPage.
21. Render table list or table grid with clear AVAILABLE and OCCUPIED states.
22. Store selected table in component state or a small POS store.
23. When a table is selected, show it clearly in the cart panel.

Test checkpoint

- Tables load from backend.
- Selecting a table updates the UI.
- Creating a sale with a selected table sends tableId instead of paymentMethod.

Notes

- Use different colors for AVAILABLE, OCCUPIED, RESERVED, and CLEANING if the backend already returns them.

8. Active table session

Backend endpoints used

Endpoint	Purpose
GET /api/tables	Identify currentSale for occupied tables.
GET /api/sales/:saleId/invoice	Load active sale details for the selected table.
POST /api/sales	Append new items to the same table session using tableId.

Implementation steps

24. When the user clicks an occupied table, read currentSale from the table object.
25. Fetch invoice or sale data for that currentSale.
26. Display existing order items so the cashier can continue the table session.
27. Keep new items separate from already-open items in the UI so only newly-added items are posted back to the backend.

Test checkpoint

- Occupied table opens its active sale.
- Existing items are visible.
- Adding new items to the same table appends to the sale instead of creating a new one.

Notes

- This is where many POS frontends go wrong. Do not resend the full open sale as if it were a new cart.

9. Payment flow

Backend endpoints used

Endpoint	Purpose
POST /api/sales/:saleId/pay	Record partial or full payments.
POST /api/tables/:tableId/close	Optional final close endpoint if your frontend uses it.

Implementation steps

28. Create a payment modal or side panel.
29. Show grand total, paidAmount, and balanceAmount for the selected sale.
30. Call the pay endpoint with amount and paymentMethod.
31. Support partial payment and then final payment.

Test checkpoint

- Partial payment changes sale status to PARTIALLY_PAID.
- Final payment changes sale status to COMPLETED.
- Table becomes AVAILABLE after full payment for active table sales.

Notes

- For takeaway, payment is often immediate. For tables, payment usually comes after the dining session.

10. Invoice and receipt view

Backend endpoints used

Endpoint	Purpose
GET /api/sales/:saleId/invoice	Load printable invoice/receipt data.

Implementation steps

32. Call the invoice endpoint after sale creation or payment.
33. Render a clean printable receipt panel or modal.
34. Show invoice number, items, totals, payments, cashier, and reservation if present.

Test checkpoint

- Invoice page or modal renders without missing fields.
- Totals match the cart and backend sale.
- Paid and balance values are clearly visible.

11. Reservations UI

Backend endpoints used

Endpoint	Purpose
GET /api/reservations	List reservations.
POST /api/reservations	Create reservation.
POST /api/reservations/:id/seat	Seat reservation.
POST /api/sales	Start sale with reservationId and tableId.

Implementation steps

35. Create a reservation list page or panel.
36. Allow creating a reservation with table, customer name, phone, guest count, date/time, and notes.
37. Allow seating a reservation when guests arrive.
38. Pass reservationId into sale creation when starting the table session from a seated reservation.

Test checkpoint

- Reservation can be created.
- Reservation status can change to SEATED.
- Sale created from reservation is linked correctly.

Notes

- Reservation is an extension of table flow, not a replacement for it.

12. Discounts and coupons

Backend endpoints used

Endpoint	Purpose

POST /api/sales	Sale creation with discountType, discountValue, or couponCode.
GET /api/coupons	Optional admin view for available coupons.

Implementation steps

39. Add discount type selector and discount input in the cart or payment area.
40. Add coupon code field.
41. Send discountType, discountValue, or couponCode to the sale endpoint.
42. Show updated totals from the backend response.

Test checkpoint

- Manual discount applies correctly.
- Coupon validates correctly.
- Grand total on the frontend matches the backend.

Notes

- If both coupon and manual discount are supplied, use the backend rule already implemented.

13. Refunds

Backend endpoints used

Endpoint	Purpose
POST /api/sales/:saleId/refund	Process full or partial refund.

Implementation steps

43. Create refund UI inside invoice or sale history view.
44. Allow full refund or partial item refund.
45. Send refund request with reason and optional item list.
46. Refresh invoice or sale details after refund.

Test checkpoint

- Refund succeeds and UI reflects updated totals.
- Inventory is restored in the backend.
- Refund history is visible if exposed in invoice or sale detail view.

14. Kitchen and reports

Backend endpoints used

Endpoint	Purpose

GET /api/kitchen/dashboard	Kitchen summary.
GET /api/kitchen/queue	Kitchen queue.
GET /api/reports/*	Analytics endpoints for admin UI.

Implementation steps

47. Show kitchen order statuses or a kitchen dashboard screen if needed.
48. Optionally subscribe to realtime kitchen events after the core POS flow is stable.
49. Build reports pages only after cashier flow is complete: daily sales, payment summary, top products, low stock, monthly trend, profit, cashier performance, and hourly sales.

Test checkpoint

- Kitchen dashboard loads.
- Report pages render real backend data.
- No analytics screen blocks cashier flow if it is still unfinished.

Notes

- Kitchen and reports are operational/admin layers. Keep them separate from the main cashier screen.

15. Recommended frontend file map

```
src/
  api/
    axios.ts
    auth.api.ts
    products.api.ts
    sales.api.ts
    tables.api.ts
    reservations.api.ts
    reports.api.ts
  store/
    auth.store.ts
    cart.store.ts
    pos.store.ts
  pages/
    LoginPage.tsx
    PosPage.tsx
    ReservationsPage.tsx
```

```
    ReportsPage.tsx
    KitchenPage.tsx
  components/
    ProductCard.tsx
    CartPanel.tsx
    CartItemRow.tsx
    TableCard.tsx
    PaymentModal.tsx
    InvoiceModal.tsx
    ReservationForm.tsx
    StatusBadge.tsx
  App.tsx
  main.tsx
  index.css
```

16. Backend-to-frontend feature map

Backend feature	Frontend screen/component	Frontend action
Auth login	LoginPage	Store token and redirect.
Products	Product grid	Load and add to cart.
Tables	Table list/grid	Select table and open session.
Sales	Cart create action	Create takeaway or table sale.
Payments	Payment modal	Record partial/full payment.
Invoice	Invoice modal/page	Display receipt details.
Reservations	Reservation page	Create, seat, and link to sale.
Coupons	Discount/coupon area	Apply promo pricing.
Refunds	Invoice or sale history	Process returns/refunds.
Kitchen	Kitchen page or badges	View and update order progress.
Reports	Reports dashboard	Display business analytics.

17. Testing checklist by milestone

Milestone A: Foundation - Login works, protected route works, no console errors.

Milestone B: Cart - Cart totals and quantity logic are correct.

Milestone C: Takeaway sale - Cart creates completed sale and clears correctly.

Milestone D: Table sale - Selected table becomes occupied after dine-in sale.

Milestone E: Active table - Occupied table reopens and accepts more items.

Milestone F: Payment - Partial and final payment work and table is released.

Milestone G: Invoice - Invoice view matches backend summary exactly.

Milestone H: Reservation - Reservation can be created, seated, and linked to sale.

Milestone I: Discount/Coupon - Frontend totals match backend totals after pricing rules.

Milestone J: Refund - Refund changes sale totals and stock correctly.

18. Recommended immediate next action

Start the frontend at Phase 5 only if Phases 1 to 4 already work in the browser. Since the current implementation already reached basic POS shell, products, cart, and takeaway sale creation, the next correct frontend step is the table UI followed by active table session handling.

Do not continue writing random screens. Build in this order: table UI → active table session → payment modal → invoice view → reservation flow.